

**Федеральное государственное образовательное учреждение высшего
образования
Национальный исследовательский технологический университет «МИСИС»
Институт компьютерных наук**

**Курс «Автоматизация технологических процессов»
Лабораторная работа №3
на тему: «Прогнозирование показателей концентрации метана,
собранные подземными датчиками угольной шахты»**

Выполнила: студент группы БИВТ-21-4

Савенко Е.И.

Проверил: Трофимов Владимир Борисович

г. Москва

2024 год

ВВЕДЕНИЕ

Добыча угля связана с работой в экстремальных и опасных условиях, включая риск взрывов метана в подземных шахтах. Для защиты шахтеров используются системы активного мониторинга, которые выявляют опасные концентрации метана и предотвращают взрывы, собирая и анализируя данные в реальном времени.

Одним из наиболее эффективных методов прогнозирования являются нейронные сети, обладающие способностью к обучению и адаптации, что позволяет им учитывать множество факторов и выдавать точные прогнозы. В данной работе рассматривается применение нейронных сетей для мониторинга и прогнозирования концентрации метана на основе данных, собранных в подземной угольной шахте.

Целью является разработка эффективного метода прогнозирования для предсказания уровня концентрации метана «ММ256» в шахтном стволе на 15 минут.

1. Исследование данных

Для начала нужно загрузить датасет и проверить основные характеристики.

Импорт библиотек и загрузка данных

Импортируем необходимые библиотеки

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
import torch
from torch.utils.data import Dataset
from torch.utils.data import DataLoader
from torch import nn
import seaborn as sns
```

Датасет состоит из 9199930 строк и 34 столбцов, первые 6 из которых являются датой и временем, а остальные указывают концентрацию веществ в шахте, к датчикам относятся: 'AN311', 'AN422', 'AN423', 'TP1721', 'RH1722', 'BA1723', 'TP1711', 'RH1712', 'BA1713', 'MM252', 'MM261', 'MM262', 'MM263', 'MM264', 'MM256', 'MM211', 'CM861', 'CR863', 'P_864', 'TC862', 'WM868', 'AMP1_IR', 'AMP2_IR', 'DMP3_IR', 'DMP4_IR', 'AMP5_IR', 'F_SIDE', 'V'.

Все параметры являются числами, а предсказываемая величина непрерывна, перед нами задача регрессии.

2. Подготовка датасета

Делаем предобработку данных.

Выбираем часть данных от 2014-03-29 13:44:57 до 2014-04-08 13:15:56.

```
# границы диапазона
start = pd.Timestamp(year=2014, month=3, day=29, hour=13, minute=44,
second=57)
end = pd.Timestamp(year=2014, month=4, day=8, hour=13, minute=15,
second=56)

timestamps = pd.to_datetime(df[['year', 'month', 'day', 'hour', 'minute',
'second']])

# фильтр по диапазону
df = df.loc[(timestamps >= start) & (timestamps <= end)]

# сброс индексов
df = df.reset_index(drop=True)

df.head()
```

Проверим данные на пропуски и отрицательные значения.

```
df.isna().sum()
df.min()
```

Видим, что пропусков нет, а отрицательные значения есть. Поскольку концентрация вещества не может быть отрицательной, можно сделать вывод, что датчик сбоил. Удалим такие строки.

```
df = df[df >= 0].dropna()
df.min()
```

Очистка от выбросов

```
def plot_for_MM256(df):
    sec_from_start = range(len(df))
    plt.figure(figsize=(15, 5))
    plt.plot(sec_from_start, df['MM256'], color = 'black', alpha=0.5)
    plt.xlabel('Номер наблюдения')
    plt.ylabel('Концентрация метана в процентах')

plot_for_MM256(df)
plt.title('Изначальные данные для MM256')
```

Видим большой выброс в районе 400 000-ой секунды, а также выброс в районе 800 000-ой секунды (рис. 1).

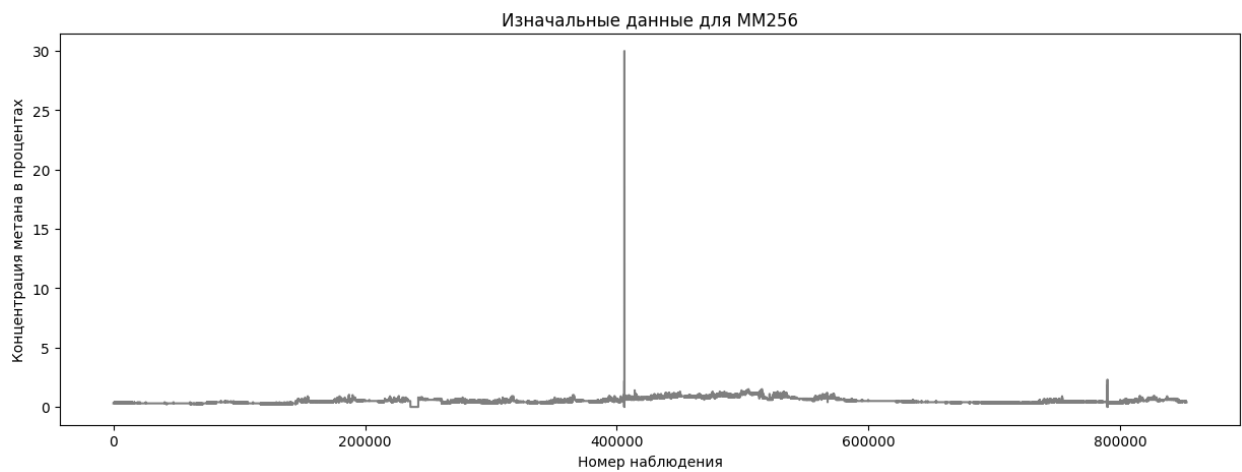


Рисунок 1. Изначальные данные для MM256.

После удаления первых выбросов график изменился (рис. 2).

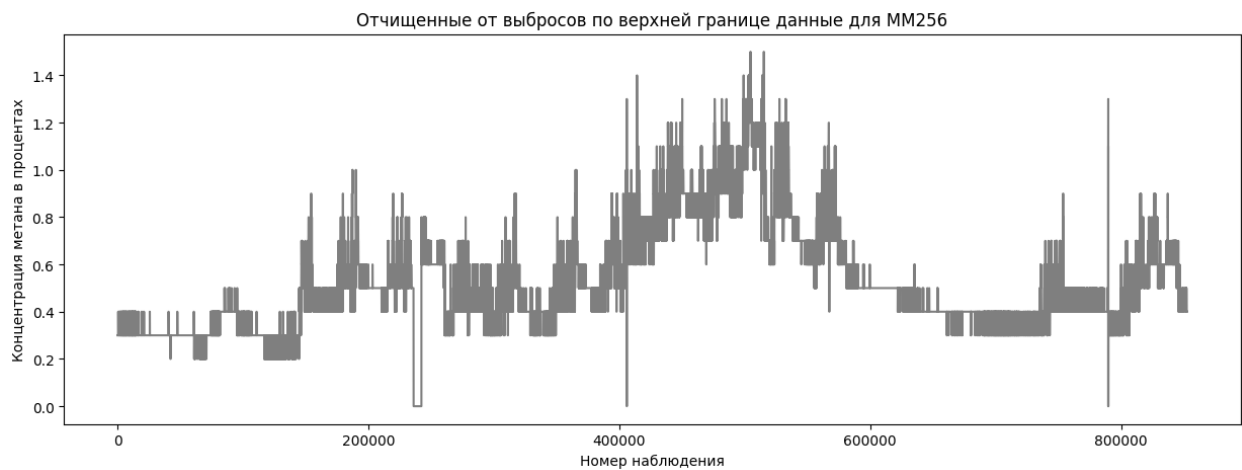


Рисунок 2. Отчищенные от выбросов по верхней границе данные для MM256.

Удалим выбросы, используя фильтр на верхнюю границу установленную в 1.6% и на нижнюю границу установленную в 0.2%. В части 800 000 секунды график очень выбивается из средних значений. Удалим и его.

```
df = df.drop(df[(df['MM256'] > 1.6)].index)
df = df.drop(df[(df['MM256'] < 0.2)].index)
df = df.drop(df[(df.index > 700000) & (df['MM256'] > 1) ].index)

plot_for_MM256(df)
plt.title('Отчищенные от выбросов данные для MM256')
```

Теперь данные не имеют значительных выбросов (рис. 3).

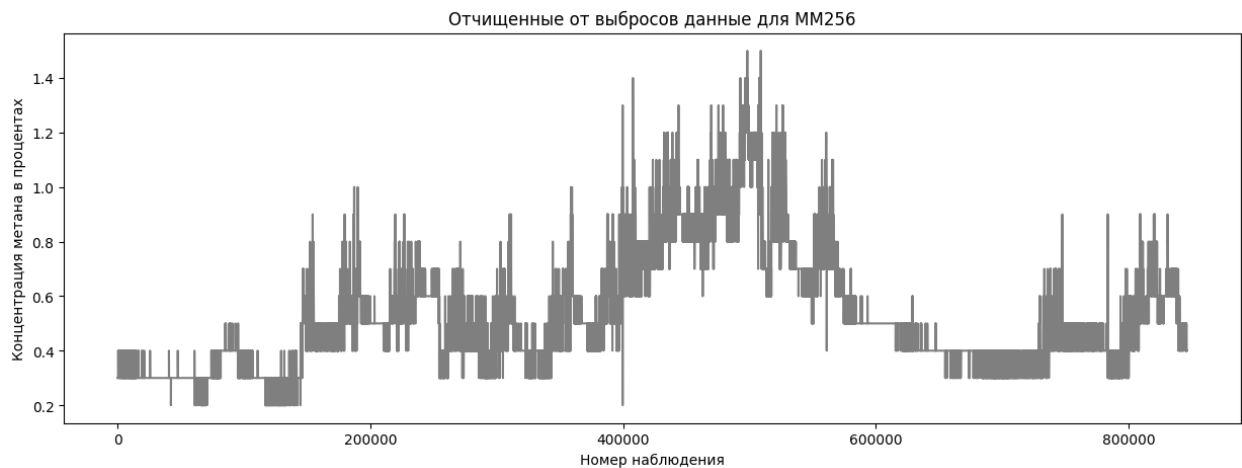


Рисунок 3. Отчищенные от выбросов данные для MM256.

Сглаживаем выборку с помощью экспоненциального фильтра

Экспоненциальный фильтр

$$X_{\text{ср}}(i) = X_{\text{ср}}(i - 1) + \alpha \cdot (X(i) - X_{\text{ср}}(i - 1)),$$

где $X_{\text{ср}}(i)$ – сглаженный сигнал; α – коэффициент фильтрации $\in (0,1)$.

Для простого сглаживания сигнала экспоненциальным фильтром можно использовать функцию библиотеки pandas.

Выборку сглаживаем дважды.

```
df_filtered = df.copy()
df_filtered['MM256'] = df_filtered['MM256'].ewm(alpha=0.1).mean()
df_filtered['MM256'] = df_filtered['MM256'].ewm(alpha=0.01).mean()
df = df_filtered.copy()
```

График.

```
df_size = len(df)

sec_from_start = range(len(df))
plt.figure(figsize=(12, 6))

plt.plot(range(df_size), df['MM256'], label='Исходные данные', alpha=0.2)
plt.plot(range(df_size), df_filtered['MM256'], linewidth=0.5, color =
'black', label='Сглаженные данные')

plt.xlabel('Номер наблюдения')
plt.ylabel('Концентрация метана в процентах')
plt.legend()
plt.title('Данные после применения экспоненциального фильтра')

plt.show()
```

После сглаживания данные стали больше похожи на линию (рис. 4).

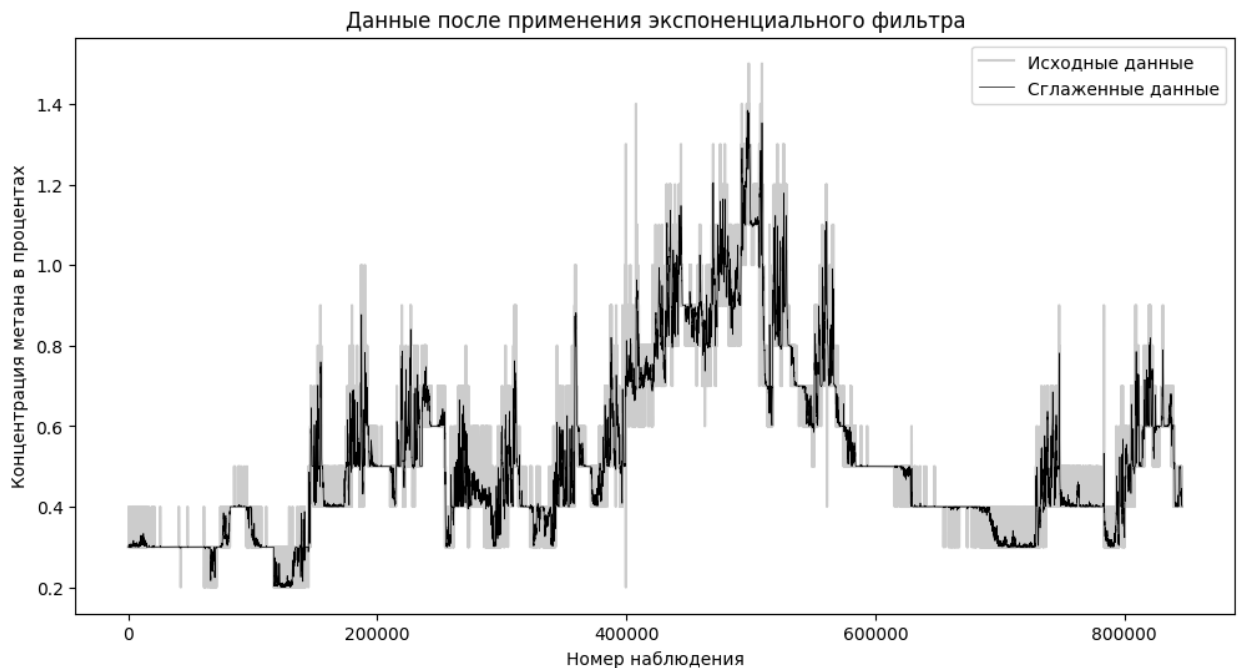


Рисунок 4. Данные после применения экспоненциального фильтра

Удалим ненужные столбцы

Удаляем столбцы времени, поскольку они не будут являться факторами, влияющими на выходную переменную

```
data = df.copy()
data = data.drop(['year', 'month', 'day', 'hour', 'minute', 'second'],
axis=1)
```

Проверим корреляцию с выходной переменной MM256

```
correlations = data.corr()['MM256']

plt.figure(figsize=(8, 6))
sns.barplot(x=correlations.index, y=correlations.values,
palette='coolwarm')
plt.title('Корреляция с MM256')
plt.xlabel('Переменные')
plt.ylabel('Коэффициент корреляции')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Видим (рис. 5), что переменные AN423, MM252, MM261, MM262, P_864 имеют низкую корреляцию с выходной переменной (меньше 0.1)

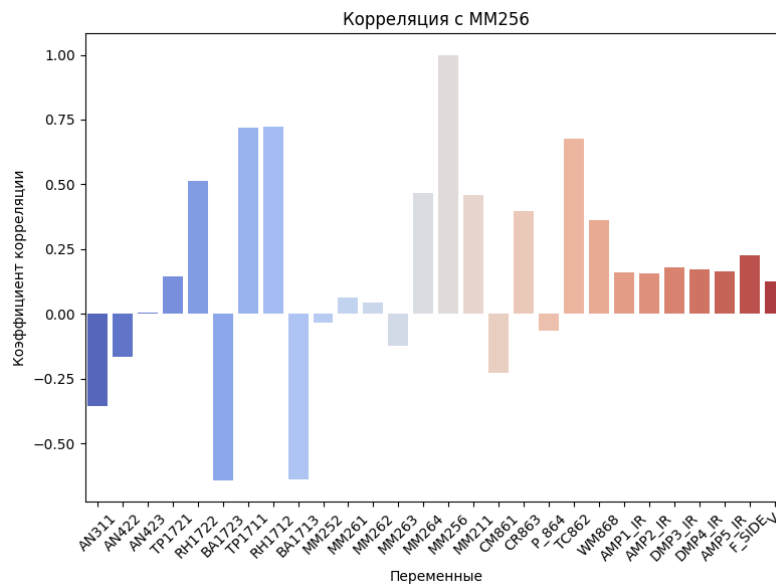


Рисунок 5. Корреляция с MM256.

Поскольку мы проверили корреляцию по критерию Пирсона, проверим также наличие линейной зависимости между факторами со слабой корреляцией.

```
weak_pearson_corr = pearson_corr[pearson_corr.abs() < 0.1].index

for col in weak_pearson_corr:
    plt.figure(figsize=(12, 6))
    sns.scatterplot(x=data[col], y=data['MM256'])
    plt.title(f'Зависимость MM256 от {col}')
    plt.xlabel(col)
    plt.ylabel('MM256')
    plt.show()
```

Составим графики зависимости (рис.6-10).

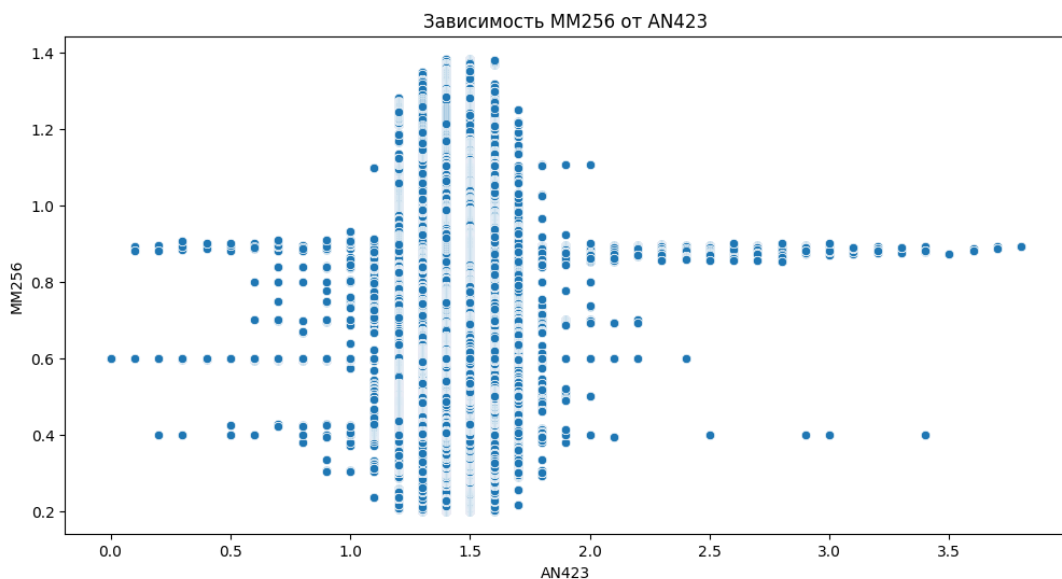


Рисунок 6. Зависимость MM256 от AN423.

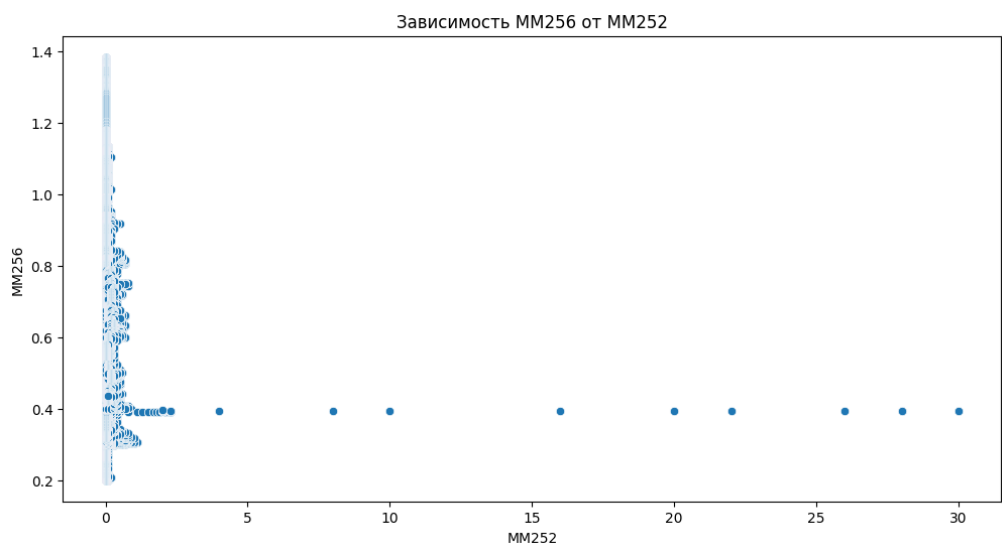


Рисунок 7. Зависимость MM256 от MM252.

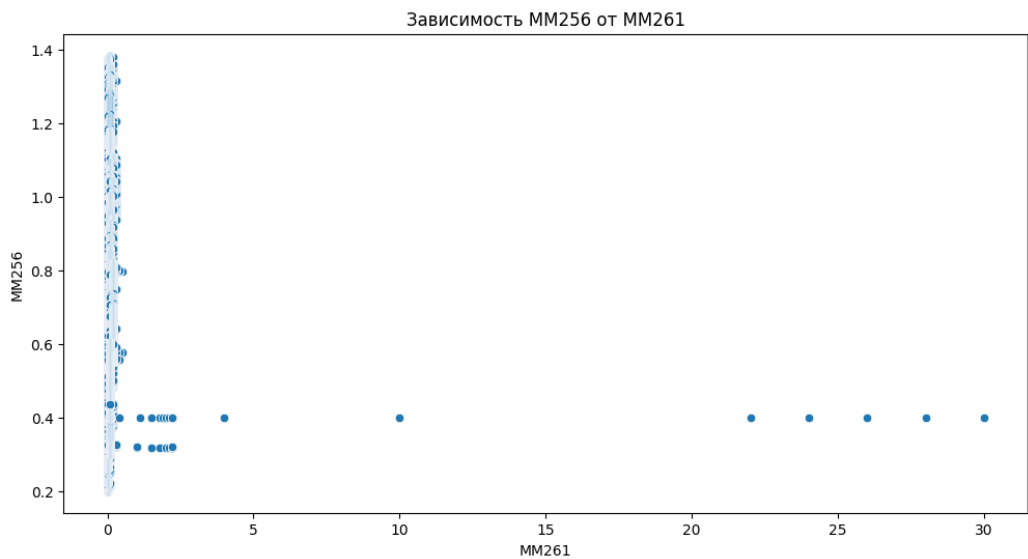


Рисунок 8. Зависимость MM256 от MM261.

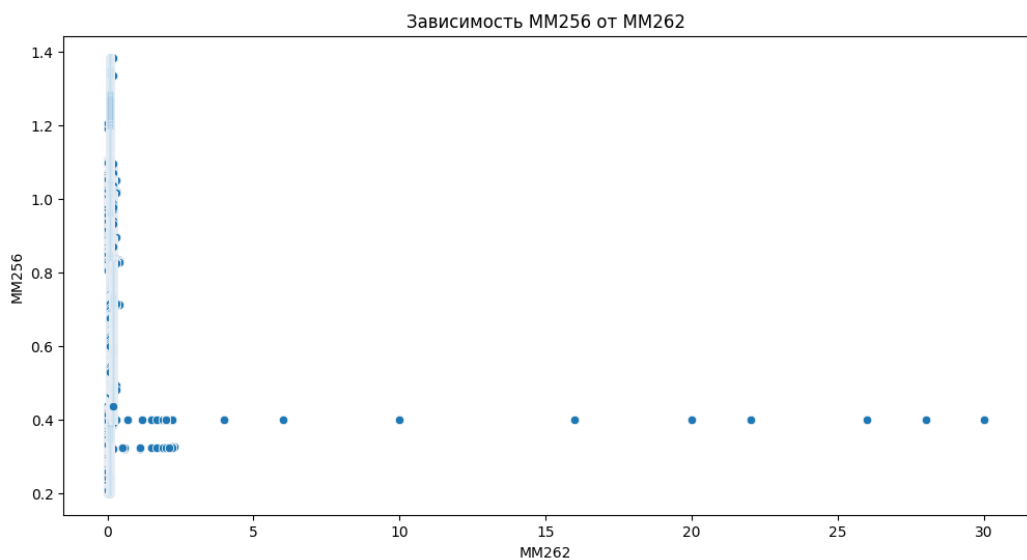


Рисунок 9. Зависимость MM256 от MM262.

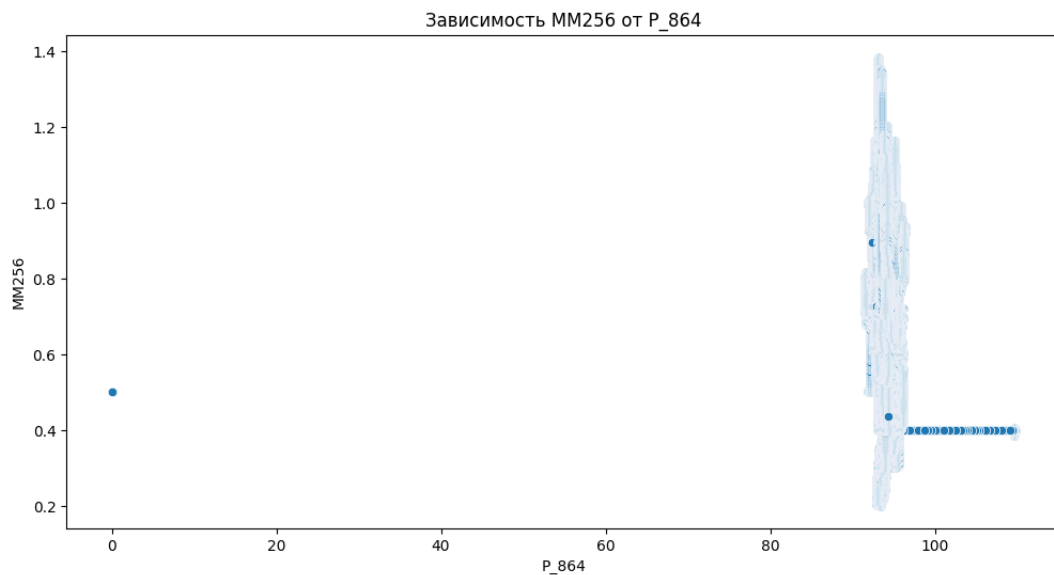


Рисунок 10. Зависимость MM256 от P_864.

По графикам видно, что о линейной зависимости речи не идет, значит оставляем все факторы.

Добавим интервал для предсказания

Добавим интервал, сдвинутый на 15 минут вперед по времени. Удалим пропущенные значения.

```
interval = 15 * 60
data['y'] = data['MM256'].shift(-interval)
data.dropna(inplace=True)
```

3. Обучение модели

Для обучения модели будем использовать PyTorch.

Разделим выборку

На входные и выходные переменные

```
X = data_ml.drop(columns='y')
y = data_ml[['y']]
```

На тестовую и обучающую

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=42)
```

Масштабирование данных

Масштабируем признаки таким образом, чтобы они находились в определенном диапазоне, обычно от 0 до 1.

```
scaler = MinMaxScaler()
X_columns = X_train.columns
y_columns = y_train.columns

X_train[X_columns] = scaler.fit_transform(X_train[X_columns])
X_test[X_columns] = scaler.transform(X_test[X_columns])

y_train[y_columns] = scaler.fit_transform(y_train[y_columns])
y_test[y_columns] = scaler.transform(y_test[y_columns])
```

Создадим свой датасет

Создадим кастомный класс для нашего датасета

```
class MethaneDataset(Dataset):
    def __init__(self, data, transform=None, target_transform=None):
        self.data = data
        self.transform = transform
        self.target_transform = target_transform

    def __len__(self):
        return self.data.shape[0]

    def __getitem__(self, idx):
        feature = self.data.iloc[idx, :-1].values.astype(np.float32)
        target = self.data.iloc[idx, -1].reshape(-1).astype(np.float32)
        if self.transform:
            feature = self.transform(feature)
        if self.target_transform:
            target = self.target_transform(target)
        return feature, target
```

Создаем экземпляры класса

```
train_data = pd.concat([X_train, y_train], axis=1)
test_data = pd.concat([X_test, y_test], axis=1)

train_dataset = MethaneDataset(train_data, transform=torch.tensor,)
test_dataset = MethaneDataset(test_data, transform=torch.tensor,)
```

Создадим даталоудер

Для того чтобы ускорить процесс обработки датасета, можно разрезать его на batch`и и обработать параллельно.

Параметр shuffle() отвечает за случайное перемешивание выборки. Отключаем его.

```
train_dataloader = DataLoader(train_dataset, batch_size=64, shuffle=False)
test_dataloader = DataLoader(test_dataset, batch_size=64, shuffle=False)
```

Собираем нейросеть

В библиотеке PyTorch по умолчанию для слоев nn.Linear не применяется автоматическая функция активации. Это означает, что если вы не явно добавили функцию активации после nn.Linear, она не будет применена. Поскольку перед нами задача регрессии, выходом должны быть непрерывные значения. Поэтому модель будет работать и без функции активации на последнем слое.

(Было проведено тестирование с функцией активации nn.ReLU() на последнем слое и без нее. Результаты оказались практически идентичными.)

```
class NeuralNetwork(nn.Module):
    def __init__(self, input_size, output_size):
        super(NeuralNetwork, self).__init__()
        self.layers = nn.Sequential(
            nn.Linear(input_size, 65),
            nn.ReLU(),
            nn.Linear(65, 65),
            nn.ReLU(),
            nn.Linear(65, 65),
            nn.ReLU(),
            nn.Linear(65, output_size),
        )

    def forward(self, x):
        logits = self.layers(x)
        return logits
```

Тренируем нейросеть

Инициализируем модель.

```
train_features, train_labels = next(iter(train_dataloader))
```

```
input_size = train_features.size()[1]
output_size = train_labels.size()[1]

model = NeuralNetwork(input_size, output_size)
```

Инициализируем параметры:

- шаг обучения
- количество эпох

```
learning_rate = 1e-3
epochs = 10
```

Функция потерь.

```
loss_fn = nn.MSELoss()
```

В качестве оптимизатора берем оптимизатор Adam.

```
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
```

Цикл обучения модели

```
def train_loop(dataloader, model, loss_fn, optimizer):
    num_batches = len(dataloader)
    train_loss, train_mae, correct, total = 0, 0, 0, 0

    for X, y in dataloader:
        # Прямой проход: вычисление предсказаний
        pred = model(X)

        # Вычисление функции потерь
        loss = loss_fn(pred, y)

        # Вычисление MAE
        mae = torch.mean(torch.abs(pred - y))

        # Обнуление градиентов
        optimizer.zero_grad()

        # Обратный проход: вычисление градиентов
        loss.backward()

        # Обновление параметров с использованием градиентного спуска
        optimizer.step()

        train_loss += loss.item()
        train_mae += mae.item()

    # Подсчет точности (Accuracy)
```

```

        correct += torch.sum(torch.abs(pred - y) <= 0.1).item()
        total += len(y)

    train_loss /= num_batches
    train_mae /= num_batches
    accuracy = (correct / total) * 100

    print(f"\nTrain Error: Avg loss: {train_loss:>8f}, Avg MAE:
{train_mae:>8f}, Accuracy: {accuracy:>0.2f}%\n")
    return train_loss, train_mae, accuracy

```

Цикл тестирования модели.

Метод `no_grad()` говорит, что через модель будут пропускаться данные, но градиент считаться не будет. Это нужно, чтобы модель не училась на тестовых данных.

```

def test_loop(dataloader, model, loss_fn):
    num_batches = len(dataloader)
    test_loss, test_mae, correct, total = 0, 0, 0, 0

    all_preds = []
    all_targets = []

    with torch.no_grad():
        for X, y in dataloader:
            pred = model(X)
            loss = loss_fn(pred, y)
            mae = torch.mean(torch.abs(pred - y))

            test_loss += loss.item()
            test_mae += mae.item()

            all_preds.append(pred)
            all_targets.append(y)

            correct += torch.sum(torch.abs(pred - y) <= 0.1).item()
            total += len(y)

    test_loss /= num_batches
    test_mae /= num_batches
    accuracy = (correct / total) * 100

    print(f"Test Error: Avg loss: {test_loss:>8f}, Avg MAE:
{test_mae:>8f}, Accuracy: {accuracy:>0.2f}%\n")

    all_preds = torch.cat(all_preds).numpy()
    all_targets = torch.cat(all_targets).numpy()

    return test_loss, test_mae, all_preds, all_targets

```

Сам цикл обучения

```
def training_cycle(model, epochs, loss_fn, optimizer):
    for t in range(epochs):
        print(f"Epoch {t+1}\n-----")
        train_loop(train_dataloader, model, loss_fn, optimizer)
        test_loop(test_dataloader, model, loss_fn)

    print("Done!")
training_cycle(model, epochs, loss_fn, optimizer)
```

Результат (рис. 11, 12). Точность около 96% говорит о том, что модель достаточно хорошо предсказывает целевую переменную на новых данных. Ошибка по функции потерь и Средняя абсолютная ошибка (MAE) также показывают, что модель в целом хорошо обучена и достаточно точно предсказывает целевую переменную.

```
Epoch 1
-----

Train Error: Avg loss: 0.002931, Avg MAE: 0.033966, Accuracy: 92.17%

Test Error: Avg loss: 0.002610, Avg MAE: 0.033248, Accuracy: 92.87%

Epoch 2
-----

Train Error: Avg loss: 0.002388, Avg MAE: 0.030729, Accuracy: 93.59%

Test Error: Avg loss: 0.002376, Avg MAE: 0.029503, Accuracy: 93.23%

Epoch 3
-----

Train Error: Avg loss: 0.002238, Avg MAE: 0.029330, Accuracy: 94.04%

Test Error: Avg loss: 0.002343, Avg MAE: 0.030776, Accuracy: 93.60%

Epoch 4
-----

Train Error: Avg loss: 0.002126, Avg MAE: 0.028379, Accuracy: 94.42%

Test Error: Avg loss: 0.002061, Avg MAE: 0.027842, Accuracy: 94.69%

Epoch 5
-----

Train Error: Avg loss: 0.002018, Avg MAE: 0.027634, Accuracy: 94.81%

Test Error: Avg loss: 0.001935, Avg MAE: 0.026776, Accuracy: 95.04%
```

Рисунок 11. Начальные эпохи обучения модели.

```

Epoch 6
-----

Train Error: Avg loss: 0.001922, Avg MAE: 0.026997, Accuracy: 95.10%

Test Error: Avg loss: 0.001833, Avg MAE: 0.026293, Accuracy: 95.38%

Epoch 7
-----

Train Error: Avg loss: 0.001852, Avg MAE: 0.026497, Accuracy: 95.33%

Test Error: Avg loss: 0.001706, Avg MAE: 0.025395, Accuracy: 95.81%
|
Epoch 8
-----

Train Error: Avg loss: 0.001788, Avg MAE: 0.026025, Accuracy: 95.52%

Test Error: Avg loss: 0.001641, Avg MAE: 0.024389, Accuracy: 96.11%

Epoch 9
-----

Train Error: Avg loss: 0.001733, Avg MAE: 0.025564, Accuracy: 95.73%

Test Error: Avg loss: 0.001612, Avg MAE: 0.024290, Accuracy: 96.12%

Epoch 10
-----

Train Error: Avg loss: 0.001686, Avg MAE: 0.025162, Accuracy: 95.91%

Test Error: Avg loss: 0.001578, Avg MAE: 0.024147, Accuracy: 96.29%

Done!

```

Рисунок 12. Конечные эпохи обучения модели.

(Результат цикла обучения с функцией активации `nn.ReLU()` на последнем слое

```

Epoch 1
-----

Train Error: Avg loss: 0.002973, Avg MAE: 0.034177, Accuracy: 92.12%

Test Error: Avg loss: 0.002601, Avg MAE: 0.032949, Accuracy: 92.79%

```

Рисунок 11*. Начальные эпохи обучения модели.

Видим, что значения схожи)

Визуализация для изначальных данных и для предсказанных

Делаем предсказания для всего датасета.

```

full_data = pd.concat([train_data, test_data], axis=0)
full_data.sort_index(inplace=True)

```

```

full_dataset = MethaneDataset(full_data, transform=torch.tensor,)

full_dataloader = DataLoader(full_dataset, batch_size=64, shuffle=False)
test_loss, test_mae, all_preds, all_targets = test_loop(full_dataloader,
model, loss_fn)

```

Денормирование данных

```

all_preds = scaler.inverse_transform(all_preds.reshape(-1, 1)).flatten()
all_targets = scaler.inverse_transform(all_targets.reshape(-1,
1)).flatten()

```

График

```

def plot_results(all_targets, all_preds):
    plt.figure(figsize=(12, 6))
    plt.plot(all_targets, label='Сглаженные значения', color = 'black',
alpha=0.2)
    plt.plot(all_preds, linewidth=0.5, color = 'black',
label='Прогнозируемые значения')
    plt.xlabel('Номер наблюдения')
    plt.ylabel('Концентрация метана по MM256')
    plt.legend()
    plt.title('Сравнение сглаженных и прогнозируемых значений')

    plt.show()

plot_results(all_targets, all_preds)

```

Визуализация результата обучения (рис. 13).

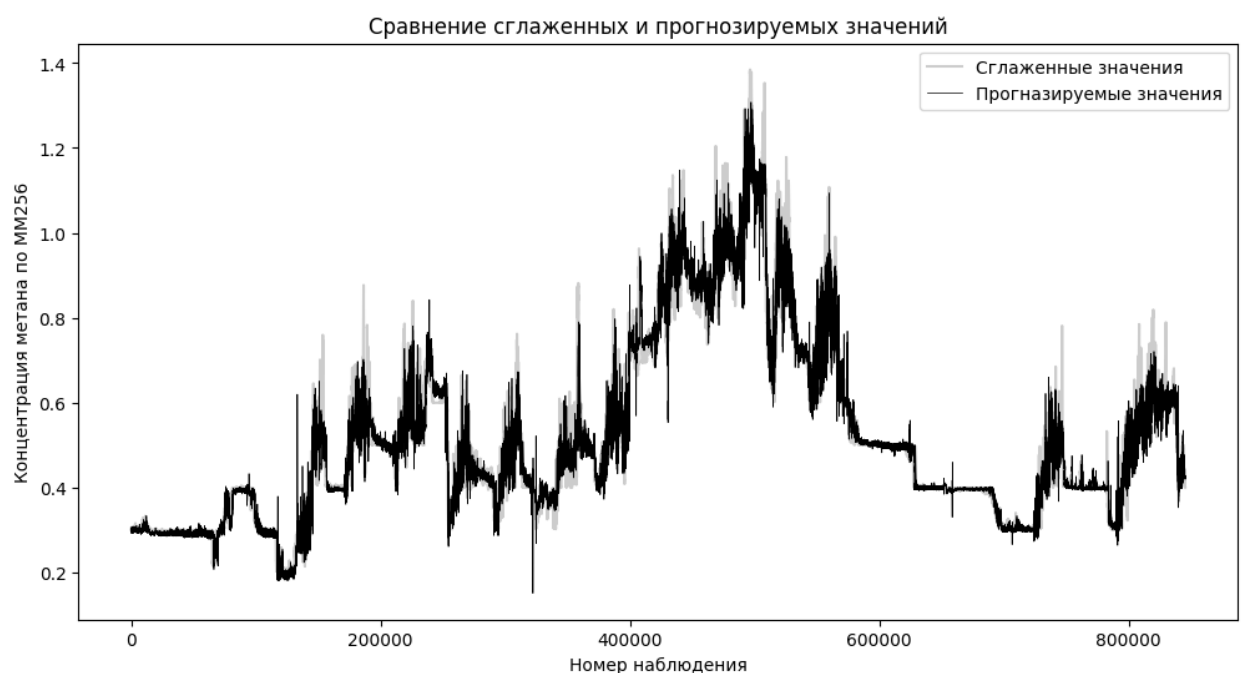


Рисунок 13. Визуализация результата обучения модели.

(Результат визуализации обучения с функцией активации `nn.ReLU()` на последнем слое



Рисунок 13*. Визуализация результата обучения модели.

Видим, что графики схожи)

ЗАКЛЮЧЕНИЕ

В результате работы была разработана модель для прогнозирования концентрации метана на основе данных, собранных подземными датчиками в угольной шахте. В первую очередь была проведена тщательная предобработка данных, включая удаление выбросов и сглаживание временных рядов с использованием экспоненциального фильтра. Эти меры позволили улучшить качество данных и подготовить их для обучения модели.

Для обучения модели была выбрана нейронная сеть на основе фреймворка PyTorch. Модель состояла из нескольких слоев, включая линейные и активационные функции, что позволило ей извлекать сложные зависимости из данных. Обучение проводилось на разделенных на обучающую и тестовую выборки данных с использованием стандартных методов оценки ошибки.

Оценка модели на тестовом наборе данных показала хорошую точность предсказаний концентрации метана. Использовались метрики средней абсолютной ошибки (MAE) и среднеквадратичной ошибки (MSE), которые подтвердили адекватность модели. Для окончательной оценки результатов была выполнена денормализация предсказанных значений, чтобы получить исходные значения концентрации метана.

Разработанная модель демонстрирует потенциал глубокого обучения в применении к задачам мониторинга и безопасности в угольной промышленности. Подходы к предобработке данных и выбору архитектуры модели оказались эффективными для данной задачи. Дальнейшие исследования могут быть направлены на улучшение архитектуры модели и дополнительное улучшение качества предсказаний путем включения дополнительных признаков или оптимизации параметров обучения.